

Mock-тестирование

1. Проблема: тесты зависят от внешнего мира

Представьте функцию, которая:

- обращается к базе данных
- делает HTTP-запрос к внешнему API
- читает файл с диска
- отправляет email

Если тестировать её **«в лоб»**, возникают проблемы.

Что не так с «лобовым» тестированием?

Проблема	Почему это больно
HTTP-запрос занимает время	Тесты идут минуты вместо миллисекунд
Сервер API упал	Тест падает, хотя код работает
Ответ API может меняться	Тест недетерминирован
API платный, письма реальные	Деньги, спам, необратимость
Тест может удалить данные из БД	Опасно для реальных данных
Нужна тестовая БД, mock-сервер	Сложно настраивать

Вывод: тесты должны быть **быстрыми, предсказуемыми и безопасными**.

2. Решение: Mock-объекты

Mock (мок) — это объект, который имитирует поведение реального объекта в контролируемых условиях.

Он используется в тестировании, чтобы **изолировать тестируемый код от внешних зависимостей**.

Мок заменяет реальный объект — базу данных, API, файловую систему — и ведёт себя **предсказуемо**, как программист ему скажет.

Что дают моки?

Мок заменяет реальную зависимость и позволяет:

- **Изолировать** тестируемый код
- **Контролировать** поведение зависимости
- **Проверять** взаимодействие с зависимостью
- **Ускорить** тесты в десятки раз
- **Сделать** тесты предсказуемыми

3. Недостатки моков

Моки — мощный инструмент, но у него есть цена.

Проблема	В чём опасность
Хрупкость тестов	Любой рефакторинг ломает тест
Ложное чувство безопасности	Мок может вести себя не так, как реальный объект
Проверка реализации	Тест проверяет <i>как</i> , а не <i>что</i>
Много моков	Плохо спроектирован код
Сложность чтения	Тест трудно читать

💡 **Игра слов:** хороший тест проверяет **что** делает код (поведение), а плохой — **как** он это делает (реализацию).

4. Когда использовать моки

✓ **Стоит мокать**

- Внешние API
- Базы данных

- Файловая система
- Отправка email
- Время и `random` — чтобы тесты были предсказуемыми

⚡ Не стоит мокать

- Свой собственный код
- Простые value-объекты
- Чистые функции
- Всё подряд «на всякий случай»

💡 Золотое правило

Мокай то, что не принадлежит программисту. Свой код тестируй по-настоящему.

5. Подключение в Python

Моки живут в **стандартной библиотеке** `unittest` — устанавливать ничего не нужно, она уже есть.

```
from unittest.mock import Mock, MagicMock, patch
```

📌 Совет

Если не знаешь, что выбрать — бери `MagicMock`.
Он умеет всё то же, что `Mock`, плюс поддерживает магические методы.
В 90% случаев используют именно его.

6. Три способа создания мока

1 Создать объект вручную

```
from unittest.mock import MagicMock
```

```
mock_db = MagicMock()
```

2 Через patch как декоратор

```
from unittest.mock import patch
```

```
@patch("my_app.requests.get")
```

```
def test_something(mock_get):
```

```
...
```

3 Через patch как контекстный менеджер

```
from unittest.mock import patch
```

```
from my_app import get_user_name
```

```
def test_get_user_name():
```

```
    with patch("my_app.requests.get") as mock_get:
```

```
...
```

⚠ Важно про путь в patch

Указывается **там, где объект используется**, а не там, где он определён.

Если `requests` импортирован в `my_app` — патчим `my_app.requests.get`, а не `requests.get`.

7. Методы: настройка поведения

return_value — задаёт возвращаемое значение

```
mock_db.get_user.return_value = {"name": "Alice"}  
result = mock_db.get_user(1)    # {"name": "Alice"}
```

side_effect — задаёт побочный эффект

```
# Возвращает по очереди  
mock.method.side_effect = [1, 2, 3]
```

```
mock.method() # 1
mock.method() # 2
mock.method() # 3

# Бросает исключение
mock.method.side_effect = ConnectionError("Сеть недоступна")
mock.method() # ConnectionError

# Вычисляет на основе аргумента
mock.method.side_effect = lambda x: x * 2
mock.method(5) # 10
```

8. Методы: проверка вызовов

Метод	Что проверяет
<code>mock.method.assert_called()</code>	Вызывался хоть раз
<code>mock.method.assert_called_once()</code>	Вызывался ровно один раз
<code>mock.method.assert_called_with(1, key="value")</code>	Последний вызов с такими аргументами
<code>mock.method.assert_called_once_with(1)</code>	Ровно один раз и с такими аргументами
<code>mock.method.assert_not_called()</code>	Не вызывался

```
mock.method.assert_called() # 
mock.method.assert_called_once_with(1) # 
mock.method.assert_not_called() # 
```

Важно

Эти методы **не возвращают** `True/False`. Они **бросают** `AssertionError`, если проверка не прошла.

9. Методы: очистка

`reset_mock()` — сбросить всю историю вызовов

```
mock.reset_mock()
```

Зачем это нужно

Чтобы один тест не «протекал» в другой.

Если мок используется в нескольких тестах, без сброса история вызовов накопится, и проверки будут давать неверный результат.

10. Итоги

Запомни главное

Мокай то, что тебе не принадлежит. Свой код тестируй по-настоящему.

- **Mock** — это объект-дублёр, который проверяет взаимодействие.
- Его главная цель — **изоляция** тестируемого кода от внешних зависимостей.
- Моки делают тесты **быстрыми, предсказуемыми и сфокусированными**.
- Использовать их нужно с умом, чтобы не сделать тесты хрупкими.